Action Pack: The View and Controller

When you think about it, the view and controller parts of MVC are pretty intimate. The controller supplies data to the view, and the controller receives events from the pages generated by the views. Because of these interactions, support for views and controllers in Rails is bundled into a single component, Action Pack.

Don’t be fooled into thinking that your application’s view code and controller code will be jumbled up because Action Pack is a single component. Quite the contrary—Rails gives you the separation you need to write web applications with clearly demarcated code for control and presentation logic.

View Support

In Rails, the view is responsible for creating all or part of a response to be displayed in a browser, to be processed by an application, or to be sent as an email. At its simplest, a view is a chunk of HTML code that displays some fixed text. More typically, you’ll want to include dynamic content created by the action method in the controller.

In Rails, dynamic content is generated by templates, which come in three flavors. The most common templating scheme, called Embedded Ruby (ERB), embeds snippets of Ruby code within a view document, in many ways similar to the way it’s done in other web frameworks, such as PHP or JavaServer Pages (JSP). Although this approach is flexible, some are concerned that it violates the spirit of MVC. By embedding code in the view, we risk adding logic that should be in the model or the controller. As with everything, while judicious use in moderation is healthy, overuse can become a problem. Maintaining a clean separation of concerns is part of the developer’s job.

You can also use ERB to construct HTML fragments on the server that can then be used by the browser to perform partial page updates. This is great for creating dynamic Hotwired interfaces. We talk about these starting in *Iteration F2: Creating a Hotwired Cart*.